

SESIÓN 8

Uso responsable de IA

Guía pedagógica para programación con criterio

Duración:	1 hora
Fase:	MARZO — Sistema base (Ingeniería pura)
Conceptos clave:	IA como herramienta, criterio, validación

Generada: 07 de April de 2026

1. Idea central de la sesión

La IA no es magia, es una herramienta imperfecta que hay que cuestionarse.

En esta sesión, el alumno aprenderá que:

- La IA genera código rápido, pero no lo entiende
- Leer y validar el código de la IA es **más importante** que generarlo
- Un buen ingeniero sabe cuándo desconfiar de la IA
- El criterio técnico es lo que diferencia usar tecnología de entenderla

Resultado esperado: El alumno sale con la capacidad de revisar código generado por IA y detectar errores.

2. Conexión con conocimientos previos

En Scratch: Tú escribías todo el código. La IA no existía en tu flujo.

En HTML/CSS: Entendías cada línea porque tú la escribías.

En JavaScript: Aprendiste qué es el estado, los eventos, la UI. Empezaste a pensar como ingeniero.

Ahora: El ingeniero sigue pensando igual, pero usa herramientas. La diferencia: DEBE validar lo que genera la herramienta.

3. Explicación clara del concepto

¿Qué es la IA en el contexto de programación?

Una IA como Claude es un modelo de lenguaje entrenado con millones de líneas de código. Puede predecir cuál es el siguiente código que escribir basándose en patrones.

Lo que hace bien:

- Generar código rápido basado en patrones conocidos
- Explicar conceptos
- Refactorizar código existente
- Ayudarte a pensar en casos borde

Lo que NO puede hacer:

- Entender el contexto de tu proyecto (aunque lo describas)
- Garantizar que el código es correcto
- Validar que respeta la arquitectura de tu sistema
- Pensar en consecuencias a largo plazo

El peligro más grande:

Un código que "funciona" pero **no debería estar así** en tu arquitectura.

4. Diseño antes de código: El flujo correcto

Flujo INCORRECTO (lo que muchos hacen):

1. "Hazme un juego de Tic Tac Toe"
2. La IA genera 300 líneas de código
3. Lo copias, funciona... a veces
4. ¿Cómo funciona? No tienes idea
5. Se rompe y no sabes arreglarlo

Flujo CORRECTO (lo que haremos):

1. **Pensar** qué necesita Tic Tac Toe (turnos, estado, victoria)
2. **Diseñar** la estructura (qué archivo, qué responsabilidad)
3. **Pedir** a la IA código para una parte específica
4. **Leer y validar** línea a línea
5. **Decidir** si entra o no en la arquitectura
6. **Refactorizar** si es necesario

La diferencia clave: El ingeniero dirige. La IA obedece.

5. Aplicación al proyecto: Tic Tac Toe

En la próxima sesión (9) diseñaremos Tic Tac Toe. Así usaremos la IA correctamente:

Paso 1: Diseñar (sin código)

- ¿Quién guarda el estado del tablero? → un array
- ¿Quién valida movimientos? → función validar()
- ¿Quién calcula ganador? → función calcularGanador()
- ¿Quién actualiza la UI? → función mostrarTablero()

Paso 2: Pedir a la IA UNA función

"Dame una función que valide si un movimiento es legal en Tic Tac Toe. Recibe: tablero (array), posición (número), jugador (X u O) Devuelve: true si es legal, false si no"

Paso 3: Revisar el código que te da

- ¿Cubre todos los casos? (¿Y si la casilla ya está ocupada?)
- ¿Usa variables claras?
- ¿Está en el archivo correcto?

Paso 4: Decidir

"Este código es correcto, lo uso" o "Necesito que sea distinto porque..."

Paso 5: Integrar en el proyecto

Copias el código, lo pones en el lugar correcto, lo pruebas.

6. Errores típicos de la IA que DEBES detectar

Error	Ejemplo	Cómo detectarlo
Código redundante	Calcula ganador 3 veces en la misma función	Lee el código y ves repetición
Lógica correcta, arquitectura mala	Calcula la lógica en un onclick de HTML	Revisa: lógica va en .js, no en HTML
Casos borde ignorados	Valida que hay ganador, pero no cuando el tablero se llena?	Preguntale: ¿qué pasa si el tablero se llena?
Variables genéricas	Usa 'a', 'b', 'temp' en lugar de nombres claros	Preguntale: ¿entiendes para qué sirve cada cosa
Duplica responsabilidad	Dos funciones hacen lo mismo	Buscas si la función ya existe en el proyecto

7. Guía: Qué pedirle a la IA (y qué NO)

■ BIEN

Prompt: "Dame una función que cuente las X en un array"

Por qué: Específico, función aislada, claramente testeable

■ BIEN

Prompt: "Explicame cómo se calcula el ganador en Tic Tac Toe"

Por qué: Pedir explicación, no código

■ BIEN

Prompt: "¿Qué casos borde me faltan para validar un movimiento?"

Por qué: Pedir crítica, no código

■ MAL

Prompt: "Hazme un juego de Tic Tac Toe completo"

Por qué: Demasiado amplio, recibes 300 líneas que no entiendes

■ MAL

Prompt: "Soluciona este bug"

Por qué: Sin contexto, la IA adivina; puede romper algo más

■ MAL

Prompt: "Refactoriza mi código para que sea más bueno"

Por qué: Ambiguo; puede cambiar la arquitectura sin que te des cuenta

8. Preguntas de pensamiento crítico

1. ¿Qué diferencia hay entre "la IA escribió mi código" y "yo escribí el código con ayuda de la IA"?
2. Si un código 'funciona pero no sé cómo', ¿puedo utilizarlo en mi proyecto? ¿Por qué?
3. La IA te da dos formas distintas de implementar la validación de movimientos. ¿Cómo eliges?
4. ¿Es más importante que el código sea corto o que sea claro?
5. Si la IA viola la arquitectura del sistema, ¿deberías usarlo aunque 'funcione'?

9. Mini-reto para 1 hora

Objetivo: Experimentar el flujo correcto de usar IA.

Tiempo: 50-55 minutos

Resultado: Código que entiendes y críticas

Actividad:

1. **Pensar (10 min):** Diseña en papel la función que valida un movimiento en Tic Tac Toe. ¿Qué recibe? ¿Qué devuelve? ¿Qué casos controla?
2. **Pedir (5 min):** Escribe un prompt específico para la IA pidiendo esa función. (No: "Hazme el validador". Sí: "Dame una función que reciba [X] y devuelva [Y]")
3. **Revisar (15 min):** La IA te da el código. Ahora:
 - Lee línea a línea
 - Pruébalo mentalmente con casos: movimiento legal, casilla ocupada, fuera del tablero
 - Escribe si es correcto o qué falta
4. **Criticar (10 min):** Pregunta a la IA:
 - "¿Qué casos borde no cubres?"
 - "¿Hay forma más clara de escribir esto?"
5. **Decidir (5 min):** ¿Usas este código o pides una versión distinta? Escribe tu decisión y por qué.

10. Antipatrones: Qué NO hacer

■ Copiar código sin leerlo

Es exactamente como suspender un examen porque encuentras las respuestas online. No aprendes, y cuando falla, no sabes qué arreglar.

■ Asumir que 'funciona' = 'está bien'

Un código puede funcionar en el caso feliz pero romper en casos borde. Tienes que pensar: ¿qué puede salir mal?

■ Usar la IA para esquivar el pensamiento

Si generas código sin antes haber pensado en la arquitectura, el código será caótico.

■ No respetar la separación de responsabilidades

La IA podría poner lógica en HTML, o mezclar dos responsabilidades. Tienes que detectar y corregir.

■ Aceptar explicaciones vagas de la IA

Si la IA te explica algo y no lo entiendes, pregunta de nuevo. Si sigue sin entender: busca en Internet.

11. Resultado esperado al final de la sesión

El alumno debe salir con:

- ✓ Entendimiento de qué puede y qué no puede hacer la IA
- ✓ Capacidad de leer código generado y detectar errores
- ✓ Confianza en cuestionar el código de la IA
- ✓ Mentalidad: "IA genera, yo valido"
- ✓ Experiencia práctica de ese flujo

Indicadores de éxito:

- Puede explicar por qué el código generado es correcto o no
- Hace preguntas específicas a la IA (no genéricas)
- Detecta violaciones de la arquitectura del sistema
- No asume que código "funciona" = "está bien"

12. Estructura temporal de la sesión (60 minutos)

Fase	Tiempo	Actividad	Objetivo
Intro	5 min	Explicación: qué es la IA como herramienta	Contexto claro
Concepto	10 min	Flujo correcto vs incorrecto	Mentalidad correcta
Casos	10 min	Errores típicos + ejemplos concretos	Reconocer problemas
Práctica	30 min	Mini-reto: diseño → pedir → revisar → criticar	Experiencia
Cierre	5 min	¿Qué aprendiste? ¿Qué seguimos la próxima sesión?	Reflexión

13. Notas pedagógicas para el mentor

Contexto: Esta es la ÚLTIMA sesión antes de empezar a construir (Sesión 9). Es el momento para establecer un mindset correcto sobre IA.

Tono: No es "la IA es mala". Es "la IA es una herramienta que hay que usar con cabeza".

Clave: El alumno debe experimentar el flujo completo: pensar → pedir → revisar → criticar → decidir. No es suficiente que lo entienda teóricamente.

Riesgo común: El alumno dirá "la IA me lo hace, ¿para qué pienso?" Respuesta: "Porque el ingeniero dirige, la IA obedece. Si no piensas, la IA hará lo que quiera."

Seguimiento: En todas las sesiones siguientes, durante el mini-reto: refuerza esta mentalidad. Cada vez que use la IA, pregunta: "¿por qué confías en este código?"

Variante avanzada (si el alumno es rápido):

Pide que compare dos soluciones distintas de la IA para lo mismo. ¿Cuál es mejor? ¿Por qué? Esto desarrolla criterio real.

14. Conexión con la próxima sesión (Sesión 9)

Sesión 9: Diseñar Tic Tac Toe (sin código)

En esta sesión practicaste el flujo correcto de usar IA. En la próxima, lo aplicarás construyendo el primer juego.

Lo que harás:

1. Diseñar (en papel, pseudocódigo, diagramas)
2. Identificar qué funciones necesitas
3. Usar la IA para generar esas funciones
4. Integrar en la arquitectura existente

Por eso esta sesión es crítica: si no internalizas "pienso primero, pido después, valido tercero", la Sesión 9 será caótica.

Principio de oro: La IA es un copiloto. Tú eres el piloto. Si no sabes dónde vas, el copiloto te pierde.